

From Paper to Picture: Automated Animated Explainers for Academic Research

Deep Patel

April 2026

Motivation & Background

Academic papers are dense. Even within a subfield, researchers often struggle to quickly parse the core ideas of a new paper — let alone communicate those ideas to broader audiences. The standard artifacts of academic work (PDFs, $\text{\LaTeX}$ source, GitHub repositories) optimize for precision and reproducibility, but not for comprehension.

Animated visualizations have proven remarkably effective at explaining complex ideas. Channels like 3Blue1Brown and Manim-based explainers routinely make advanced mathematics feel intuitive to general audiences. Yet producing these animations requires significant manual effort — a researcher must first write their paper, then separately author an animation that faithfully represents it. This gap means most papers never get a visual explainer, and when they do, it is often produced by a third party who may introduce inaccuracies.

Recent advances in large language models offer a new path. LLMs can parse dense technical text, extract structured representations of key ideas, and generate code — including animation code. This project explores whether that capability can be assembled into a practical tool: one that takes an academic paper as input and produces a reviewable, author-correctable animated explainer as output. The vision is for these animations to become a standard artifact of the research process, submitted alongside papers and codebases to improve clarity, reproducibility, and public understanding of complex work.

Proposed Deliverable

The deliverable is a working prototype tool — **PaperAnimate** (name TBD) — with the following core functionality:

1. **Ingestion:** Accept a PDF or $\text{\LaTeX}$ source of an academic paper.
2. **Extraction:** Use an LLM to identify key concepts, theorems, algorithms, or figures worth animating.
3. **Generation:** Produce a [Manim](#) animation script from the extracted content.

4. **Review loop:** Allow the author to prompt corrections in natural language (e.g., “the arrow should point from A to B, not B to A”) which are applied iteratively to the animation.
5. **Export:** Render the final animation as an MP4 or GIF suitable for submission alongside a paper.

The final submission will include: the tool itself (open-source Python), a short paper describing the approach, and 2–3 example animations generated from real published papers.

First Steps

Initial exploration has focused on two questions: *can an LLM reliably extract animatable structure from a paper?* and *can it generate syntactically correct Manim code?*

Preliminary tests with GPT-4 and Claude on several ML papers (e.g., the original Transformer paper) are encouraging on both fronts. Given a section of a paper and a prompt such as “describe the key algorithmic steps in this section as a sequence of visual scenes,” the model produces coherent scene descriptions. Feeding those descriptions back with a prompt to generate Manim code yields runnable (though sometimes imperfect) animations on the first try roughly 60–70% of the time.

The main challenges identified so far:

- **Faithfulness:** Models sometimes hallucinate details or simplify in ways that distort the original meaning. Author review is therefore essential.
- **Manim complexity:** Generated code tends to use basic primitives correctly but struggles with precise layouts and timing.
- **Domain specificity:** Math-heavy papers (e.g., proof-based ML theory) require richer handling of $\text{\LaTeX}$ equations within animations.

A sample scene description extracted from a paper on attention mechanisms illustrates the approach:

Scene 1: Show three vectors Q , K , V appearing. Draw arrows from Q and K toward a dot-product symbol, yielding a scalar score. Apply softmax to produce attention weights. Animate a weighted sum of V vectors producing an output vector.

This maps cleanly to a Manim scene sequence and serves as a proof of concept for the extraction pipeline.

Resources & Tools

Resource	Purpose
Manim Community	Animation engine for generating visualizations
OpenAI / Anthropic APIs	LLM backbone for extraction and code generation
PyMuPDF / pdfplumber	PDF parsing and text extraction
arXiv API	Source of test papers for development
LangChain	Orchestrating multi-step LLM pipelines
Python 3.11+	Primary implementation language

Local GPU compute will be used to experiment with open-weight models (LLaMA 3, Mistral) as potential on-device alternatives to API-based LLMs, reducing cost and enabling fully offline use. API access may be considered if results or generation time are unfavorable.

Milestones & Timeline

Date	Milestone
April 14	Working PDF → scene-description pipeline; at least 3 test extractions
April 21	Manim code generation from scene descriptions; manual review pass
April 28	Natural language correction loop implemented; end-to-end demo working
May 5	Polish, README, 2–3 example animations rendered from real papers
May 12	Final paper and submission